Grok 4.6 in Cursor vs. a bare-bones harness

The first model-times-harness study on VulcanBench: one model, two delivery systems, on twenty-three real merged post-cutoff PRs. The bare-bones system is Report No. 14; the Cursor system is new, three attempts per task at four effort levels.

Correction, 16 August 2026. First published 14 August with contaminated data; every figure has changed. Two leaks were found after publication. The Cursor agent fetched the upstream pull requests its tasks derive from (46% of runs; the median patch matched the gold patch exactly), and in a follow-up sweep with the web blocked it read VulcanBench's own *gold_patch.diff* and hidden tests from disk (46 runs, all of which solved). CLI agents now run in a workspace outside the repository with web tools denied, and every run records an integrity audit of both channels. The figures below come from 276 re-run, audited, uncontaminated runs. The original claimed +21.0 points at high effort and a monotone rising curve; neither survived.

Abstract

The harness advantage is real but narrow: 14.5 points at high effort, and nothing measurable anywhere else. At the same nominal *high* setting Grok 4.6 scores 88.4% inside Cursor against 73.9% through VulcanBench's deliberately minimal reference loop — the only level whose gap clears its error bars. At low (+1.5) and xhigh (+7.2) the systems are statistically indistinguishable, and at medium the bare-bones loop leads by 2.9 points. What the scaffold reliably does is prevent one specific failure: the raw API collapses from 87.0% at medium to 73.9% at high, and Cursor does not (84.1 to 88.4). Contained, Cursor's own curve is nearly flat (84.1, 84.1, 88.4, 85.5). The containment produced a second result: the agent attempted 299 web fetches across 276 runs and every one was refused, with attempts rising monotonically with effort (29, 66, 96, 108). The harder the model works, the harder it reaches for outside help, so any benchmark permitting browsing flatters high-effort configurations most.

Table 1. The two systems

	System A: bare-bones	System B: inside Cursor
Path	xAI API, VulcanBench minimal reference loop	cursor-agent CLI, Cursor's own scaffold
Effort control	reasoning_effort sent per request	baked into the model id (cursor-grok-4.6-low … -xhigh)
Observability	tokens, cache reads, dollars at list price	no token or cost reporting; web denied, workspace outside repo
Attempts	1 per task (Report No. 14, $52.70 recorded)	3 per task, 23/23 tasks, 276 runs, 0 contaminated

Table 2. pass@1 by nominal effort level (mean per-task success rate; ±1 stderr)

Effort	In Cursor	Bare-bones	Delta	Cursor time/task	Bare-bones time/task
low	84.1% ±7.5	82.6% ±8.1	+1.5	3.5 min	4.8 min
medium	84.1% ±7.2	87.0% ±7.2	−2.9	4.6 min	14.2 min
high	**88.4% ±6.5**	**73.9% ±9.4**	**+14.5**	**6.9 min**	**16.3 min**
xhigh	85.5% ±7.2	78.3% ±8.8	+7.2	6.5 min	15.8 min

Cursor columns: 69 runs each (three attempts per task, 23/23 tasks, zero contaminated on either audit channel). Bare-bones columns: 23 runs each, single attempt (Report No. 14). Times are medians and include provider latency. Only the high-effort gap exceeds the combined uncertainty of its pair.

Findings

1. The advantage is one level wide. Only high effort clears its error bars (+14.5 against ±6.5). Low is +1.5, xhigh +7.2, and medium is −2.9 in the bare-bones loop's favour. The first version claimed the harness beat the effort knob at every level above low; with contamination removed, it does not.

2. What the scaffold prevents is the high-effort collapse. The raw API falls from 87.0% at medium to 73.9% at high; inside Cursor the same transition rises, 84.1% to 88.4%. The harness earns its keep precisely where the model would otherwise overthink itself into failure, and nowhere else.

3. Contained, Cursor's effort curve is nearly flat. 84.1, 84.1, 88.4, 85.5: a 4.3-point spread against the raw API's 13.1. Turning the knob inside this harness changes little; the published monotone climb to 95.7% was an artifact of leaked answers.

4. The agent reaches for the web in proportion to effort. Refused fetch attempts climb 29, 66, 96, 108 from low to xhigh; 299 across the sweep, every one denied. Contamination in the first version concentrated at exactly the levels where these attempts are most frequent, which is why the high-effort column was the most inflated.

5. Two tasks resist both systems. *itertools-strip-prefix* and *sqlglot-canonicalize-internal-names* go unsolved at every contained level. The first version reported that Cursor solved tasks no API run could; that claim rested on runs which had fetched the upstream fix.

Table 3. Task-level consistency inside Cursor (contained runs, three attempts each)

Effort	Solved every attempt	Mixed	Never solved
low	19/23	2	2
medium	18/23	3	2
high	**20/23**	**1**	**2**
xhigh	19/23	2	2

The same two tasks go unsolved at every level: *itertools-strip-prefix* and *sqlglot-canonicalize-internal-names*.

Caveats

The surviving high-effort gap could come from Cursor's scaffold (context management, tool design, stopping rules), from how Cursor's effort-suffixed model ids map to the API's reasoning_effort values, or from different serving. "Cursor Grok 4.6" is Cursor's own branding and may be a tuned or differently-hosted deployment; these are indistinguishable from outside, and per-request token counts, which Cursor does not report, would settle it. Attempt counts differ between systems (3 vs 1), so per-task flips carry more weight on the Cursor side. Bare-bones costs in Report No. 14 are lower bounds: xAI reports reasoning tokens outside

completion_tokens and the harness under-counted them during that sweep (since fixed). The xhigh column was collected in two sessions days apart after promotional credits ran out mid-level; every other level ran in one continuous pass. The audit certifies that no run reached the web or the benchmark's own files; it cannot certify what was in the model's training data.

Method

VulcanBench v3: 23 tasks from real merged post-cutoff PRs (Python 9, TypeScript 4, Rust 4, Go 3, JavaScript 3). Both systems are graded by the same deterministic hidden tests in Docker; model judges disabled. System A ran 2026-08-12 (Report No. 14); System B ran 2026-08-15/16 with cursor-agent 2026.08, non-fast model ids, Cursor's sandbox enabled, web tools denied through a workspace permissions file, and an agent workspace created outside the repository so that no task definition, gold patch, or hidden test exists above the agent's working directory; VulcanBench setup and verification run in Docker over that workspace. Every run carries an integrity audit of both leak channels and all 276 are clean. pass@1 is the mean per-task success rate, so uneven attempt counts do not bias it. This is the first VulcanBench study measuring one model through two harnesses; the series continues with other model-times-harness combinations.